Las Vegas Workloads: Verifiable Search in a Compute Market

Zach Kelling

Hanzo AI

2026

Abstract

The obvious first workload for a decentralized compute market is inference, because inference is what people want to buy. It is also close to the worst workload the design could start with, because there is no cheap way to check it.

Verifiable search is the opposite. A search job is enormous to compute and trivial to check, the check is binary, an invalid answer simply fails rather than requiring adjudication, the work is embarrassingly parallel across heterogeneous supply, and the input is public so provider selection carries no privacy constraint. Fraud is not detected — it is impossible, which removes an entire layer of protocol.

We formalise the class, contrast its market properties with inference, and work through a live example: hash-based signature search for quantum-resistant Bitcoin spends. The example is instructive beyond itself, because analysing it produces a design rule that generalises to every proof-of-work-flavoured construction, and because the block-space arithmetic shows how a scheme's container choice can dominate its cryptography.

Contents

1 The class

Definition 1 (Las Vegas workload)**.** *A job is a pair* $(\mathcal{V}, p)$ *where* $\mathcal{V}(x, w) \in \{0, 1\}$ *is a predicate computable in time negligible against the search, and each independent trial succeeds with probability* p*. The provider's task is to find any* w *with* $\mathcal{V}(x, w) = 1$*. Expected work is* $1/p$ *trials; verification cost is independent of the work performed.*

Four market properties follow directly, and together they are unusual enough to be worth listing.

Verification is free. $\mathcal{V}$ is by construction cheap. No proof system, no replication, no committee. The job sits at tier T1 of the verification ladder at zero marginal cost.

Fraud is impossible rather than detectable. A provider cannot submit a wrong answer that passes, because passing is the definition of right. There is nothing to slash for, because there is nothing a dishonest provider can gain. The worst available misbehaviour is doing nothing, which is a liveness problem and is priced by the deadline.

Sharding is free. The search space partitions cleanly. Ten thousand providers can work disjoint nonce ranges with no coordination beyond the initial assignment, and heterogeneous hardware simply contributes at whatever rate it manages.

The input is public. There is no confidential material to protect, so provider selection is unconstrained by privacy and the job can be broadcast openly. Private preimages, where the construction has them, never leave the owner's device — only the public search is outsourced.

2 Why inference is harder

Inference has no cheap $\mathcal{V}$. Given a token stream, there is no sub-linear test that it came from the model that was paid for. Options are recomputation (as expensive as the job), replication (multiplies the bill), algebraic spot-checking of the matrix products (good, but silent about the nonlinearities and about which weights were used), or a succinct proof (three to six orders of magnitude of prover overhead).

This is not an argument against building an inference market. It is an argument for not making the verification layer's first load-bearing test be its hardest case. A market that carries search volume from day one has real verified throughput while its inference verification story matures.

3 A worked example

3.1 The construction

Consider a scheme that makes a Bitcoin spend's security rest on hash preimage resistance rather than on the discrete logarithm, using only opcodes already deployed — so that coins can be protected against a Shor-capable adversary without waiting for a consensus change.

The mechanism is forced by a limit. Bitcoin's legacy script cannot compute the index-selection function of a hash-based one-time signature within its 201-opcode budget. So instead of computing which key indices a message selects, the script hardcodes the index set and the signer grinds the transaction — varying a nonce field — until its hash lands on that set. The grind is the mechanism, and it is a Las Vegas workload in exactly the sense defined above.

3.2 The arithmetic

For a scheme committing to 9 indices from 150 in a first round and 8 from the remaining 141 in a second:

Quantity	Value	Bits
$\binom{150}{9}$	8.29×10^{13}	$2^{46.2}$
$\binom{141}{8}$	3.17×10^{12}	$2^{41.5}$
Sum, rounds sequential	8.32×10^{13}	$2^{46.2}$
Product, rounds joint	2.63×10^{26}	$2^{87.8}$

Table 1: Search size under the two possible round structures.

Whether the rounds compose additively or multiplicatively is not a detail. At 1.5×10^{10} hashes per second per accelerator, the additive reading is about 1.5 h on a single device; the multiplicative reading is 5.6×10^{8} device-years. Those are not two versions of the same product. One requires no marketplace at all; the other cannot be completed by any marketplace.

3.3 The design rule

The more interesting observation concerns the adversary. In a grind-secured scheme the honest signer's work and the attacker's work are the same computation: once a spend is broadcast, its preimages are public, and an adversary wishing to redirect the output must complete an identical search before the transaction confirms. Security is therefore

$$\text{margin} \;=\; \frac{\text{honest precomputation time}}{\text{confirmation window}} \cdot \frac{\text{honest rate}}{\text{adversary rate}}, \tag{1}$$

and the second factor is where such schemes die. Against an adversary holding SHA-256 application-specific silicon, a 10^{21} grind takes a mid-size mining pool about 1,000 s and the whole network about 1 s, while ten thousand accelerators need 77 days.

Proposition 1 (Grind selection rule)**.** *Never grind a primitive for which your adversary owns dedicated silicon.*

Applied here: if the ground function is a double SHA-256 the construction is unsound in practice regardless of its combinatorics, because the entire Bitcoin mining industry is an oracle for exactly that function. If it is RIPEMD-160 composed with SHA-256 — the `HASH160` opcode — no deployed miner accelerates it, general-purpose accelerators are the fastest available hardware, and the symmetry breaks in the defender's favour. One opcode decides whether the scheme works.

3.4 Container arithmetic

A second lesson is that where a script lives can dominate what it does.

The inversion is instructive. The bare-script *spend* is cheaper, because the script was already paid for when the output was created. That is precisely the problem: this construction is insurance, and the bare container charges the entire premium on day one, on chain, permanently, whether or not the policy is ever claimed. A committed script path defers about ninety-eight percent of the cost to the claim and reduces the permanent burden on every validating node by a factor of 225.

There is a caveat that outranks the table. If the construction depends on a script-evaluation quirk that exists only in legacy script and was removed by the segregated-witness upgrade, then it cannot be relocated at all, and the size ceiling, the opcode ceiling and the grind stop being

	Bare script	Committed script path
Funding output	9,661 vB	43 vB
Unspent-output-set cost until spent	9,661 B	43 B
Spend	646 vB	2,645 vB
Lifecycle total	10,318 vB	2,699 vB
One million protected addresses	9.66 GB	43 MB

Table 2: Block-space cost of a 9,650 B script by container, for one input and one output revealing seventeen 32-byte preimages.

implementation choices and become structural. That is the first thing to establish about any such design, because it determines whether the remaining questions are about tuning or about redesign.

3.5 The construction that needs no search

Worth stating because it reframes the whole exercise: the grind exists only because legacy script could not compute the index selection. The tapscript upgrade removed both the opcode limit and the script size limit, and hash-chain one-time signatures have since been implemented in that environment as bit commitments. Such a signature commits to the message digest directly. There is no index selection, therefore no grind, therefore no search job, therefore no compute market involvement — and the construction is studied rather than bespoke.

A market should want to know this. Selling compute for a search that a better construction eliminates is not a business; recognising it and pricing the remaining genuine search workloads is.

4 Market mechanics for search

Job specification. Declare $\mathcal{V}$ as content-addressed code, the success probability p, the search domain, and a deadline. Expected cost is $1/p$ trials divided by the aggregate rate purchased.

Partitioning. Assign disjoint domain ranges. A provider can lie about which range it searched, but cannot fabricate a solution, so the worst outcome is duplicated effort rather than theft. Overlapping assignments with a modest redundancy factor are usually cheaper than the coordination needed to prevent overlap.

Settlement. Winner-take-all invites providers to hoard a solution until the last moment when a race is close, and pays nothing to the participants who bounded the risk. Proportional payment on assigned-and-attested range coverage plus a finder's bonus is better behaved, at the cost of needing a weak attestation of coverage — which is one of the few places in this class where some trust re-enters.

Pricing. Expected work is known in advance, which makes search one of the few compute workloads that can be quoted as a fixed price with a genuine distribution rather than a guess. Variance is geometric with parameter p; the market can offer a fixed-price contract and absorb the variance across jobs, which is an insurance product and a real revenue line.

5 Summary

Search is the workload where a decentralized market's verification story is free, its adjudication story is empty, and its sharding story is trivial. Starting there is not a compromise; it is the case the architecture handles perfectly, and it buys the time needed to do inference verification properly.

The example examined here also carries a warning worth generalising: a search workload created by a design limitation may evaporate when the limitation is lifted. The durable search workloads are the ones where the search is intrinsic to the problem, not an artifact of the container.